

ML4N: SSH Shell Attack Session

MARCO DI BERNARDO, MICHELE PIO LASALVIA, STEFANO LIBERATI, and DAVIDE ROTONDO, Politecnico di Torino, Italy

Secure Shell (SSH) protocols remain a primary vector for cyber-attacks targeting cloud infrastructure and IoT devices. This report presents a comprehensive analysis of 235,035 malicious SSH sessions captured within a honeypot environment, leveraging both statistical machine learning and deep learning techniques to automate intent classification and pattern recognition. We first introduce a rigorous pre-processing pipeline that includes Base64 payload decoding and TF-IDF feature extraction to transform raw shell commands into structured numerical representations. In the supervised learning phase, we evaluate Random Forest and Multi-Output Logistic Regression models, achieving a weighted F1-score of over 99% in classifying attack tactics such as Discovery and Persistence. To uncover latent attack campaigns, we apply unsupervised clustering using K-Means on an SVD-reduced feature space. This analysis reveals five distinct behavioral clusters, successfully isolating sophisticated resource hijacking campaigns and botnet "turf wars" from automated reconnaissance noise. Finally, we implement a transfer learning approach using the BERT transformer architecture. By fine-tuning a pre-trained language model, we demonstrate that deep contextual embeddings can outperform traditional statistical methods, particularly in identifying complex, multi-stage attack intents with high precision.

CCS Concepts: • **Security and privacy** → **Intrusion detection systems**; *Network security*; • **Computing methodologies** → *Supervised learning*; *Neural networks*; Unsupervised learning.

Additional Key Words and Phrases: SSH Attacks, Honeypot, Intrusion Detection, Machine Learning, Clustering, BERT, Natural Language Processing

1 Introduction

In the current landscape, SSH is the standard protocol for server access, as most systems consist of Linux-based containers or Virtual Private Server (VPS) without graphical interface. We selected this project because it mirrors real-world operational scenarios. Leveraging machine learning to detect cyber-attacks is not only a compelling and complex challenge, but it also significantly outperforms manual log analysis in terms of speed and efficiency when identifying anomalous patterns.

The dataset includes 235,035 entries, each representing a complete session of commands executed by attackers within a honeypot environment. The report is organized into four main parts: section 2. Exploratory Data Analysis; section 3. Supervised Learning Experiments; section 4. Clustering Techniques; section 5. Neural Networks Implementations.

2 Data exploration and pre-processing

In this section, we will analyze the dataset containing all the SSH attack sessions. Through systematic data processing and temporal analysis, we characterize the timing of attacks. By analyzing the seasonal distribution, we identify specific trends where malicious activity is most concentrated.

2.1 Data exploration

From Figure 1, we can deduce a detailed timeline of daily attack activity over a nine-month period, beginning in June 2019 and concluding in early March 2020. For the first four months, the environment remained relatively quiet. The daily frequency of attacks was negligible, consistently hovering near the baseline and rarely exceeding a few hundred incidents. The situation changed in mid-October. The graph shows a sudden "step up" in activity,

Authors' Contact Information: Marco Di Bernardo, marco.dibernardo@studenti.polito.it; Michele Pio Lasalvia, michele.lasalvia@studenti.polito.it; Stefano Liberati, stefano.liberati@studenti.polito.it; Davide Rondo, davide.rondo@studenti.polito.it, Politecnico di Torino, Torino, Italy.

where the volume stabilized at a new, higher plateau of approximately 1,500 to 2,000 attacks per day. This culminated in the most significant event in the dataset: The December Spike. In early December, the frequency surged exponentially, peaking at a staggering more than 6,000 attacks in a single day. This represents a massive concentrated effort or a major systemic event, followed by an equally rapid decline as the year drew to a close. As the beginning of the year 2020, the pattern changes from a sustained plateau to extreme instability. While the overall volume stayed lower than the previous peak, the data shows aggressive, short-lived “pulses”. In late January and early February, we see several sharp spikes followed by immediate drops to near zero, arriving in the late February with a secondary resurgence, indicating that the threat or activity remained active and unpredictable leading into the spring.

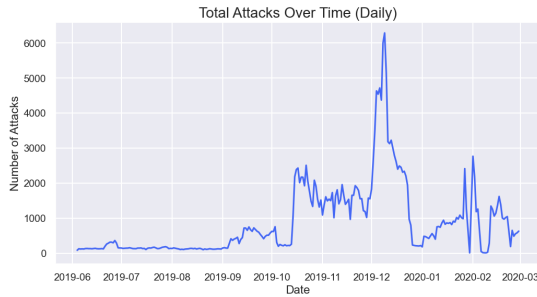


Fig. 1. Number of attacks frequencies over time.

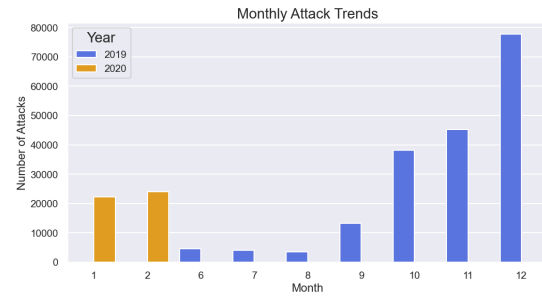


Fig. 2. Monthly attack trends.

We can deduce from the Figure 1 that while the system was stable throughout the summer of 2019, it underwent a fundamental shift in the end of the year. The massive surge in December is the defining characteristic of this period, likely representing a coordinated campaign or a major breach event, while the erratic behavior in early 2020 suggests an ongoing, albeit less organized, state of conflict or activity.

Building on the daily data, the monthly breakdown shown in the Figure 2 confirms a massive shift in scale between 2019 and 2020. From June to August 2019, activity was at its lowest, with monthly total remaining under 5,000 attacks. From September the activities surged exponentially, culminating in a massive December peak of over 77,000 attacks, the highest in the entire dataset. While daily spikes became more erratic in 2020, the monthly totals for January and February remained significantly higher than the original 2019 baseline.

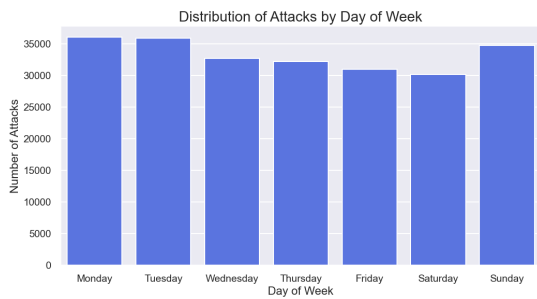


Fig. 3. Distribution of attacks by day of the week.

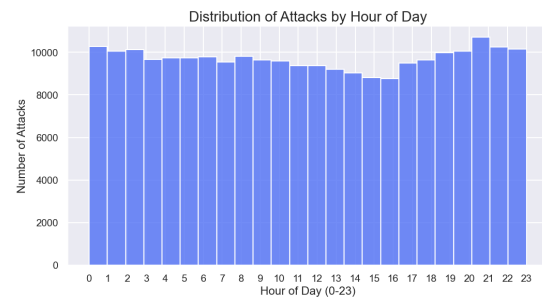


Fig. 4. Distribution of attacks by hour of day.

The weekly analysis, illustrated in Figure 3, shows a relatively consistent spread of activity with a slight bias toward the beginning and end of the week. Both Monday and Tuesday see the highest activity, both exceeding

35,000 attacks. Wednesday through Saturday shows a steady, gradual decline going from 35,000 to 30,000 attacks which represent the lowest volume of the week. Sunday activity rises significantly from the Saturday low, nearly matching the early-week peak with 34,000 attacks.

Regarding activities by time of day (Figure 4), we can clearly say that attacks remain remarkably consistent across the 24-hour cycle, yet it exhibits a distinct “crescent” pattern characterized by higher activity during the night and early morning. The data shows that the absolute peak occurs during the late evening at 21:00 PM, where volume exceeds 10,000 incidents, followed closely by a high-intensity period that last from midnight to 2:00 AM. Conversely, the system experiences its most significant period of relative calm during the afternoon, specifically between 15:00 and 16:00 PM, when attack number drop to their daily floor. Following this afternoon trough, there is a clear and steady escalation in activity that begins at 17:00 PM and climbs hourly until it reaches the nightly maximum.

The distribution of characters (Figure 5) per session is heavily skewed toward low-volume interactions, with the vast majority of data points concentrated near the zero. By utilizing a logarithmic scale on the y-axis, the graph shows that while short sessions occur with a frequency exceeding 10^5 , there is a extended range of much larger, infrequent sessions. Rare but significant outliers exist at the far edge of the spectrum, reaching up to 200,000 characters. Additionally, we can notice a small secondary group where the numbers of character are visible between 30,000 and 50,000, suggesting a distinct subset of sessions that involves more content than the standard interaction.

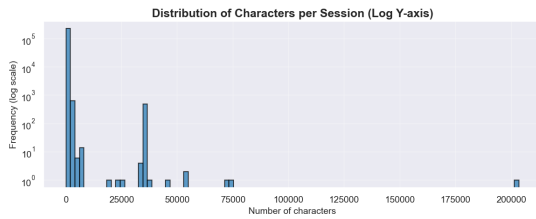


Fig. 5. Distribution of character per session.

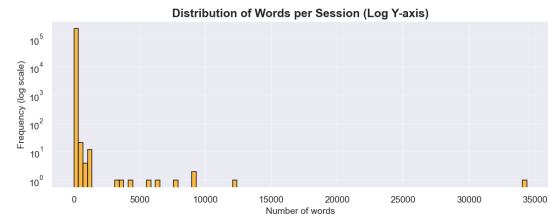


Fig. 6. Distribution of words per session.

The analysis of words per session (Figure 6) follows a nearly identical pattern to the character distribution, again utilizing a logarithmic scale to capture data across a vast range of frequencies.

The data is overwhelmingly dominated by extremely brief sessions, with the highest concentration occurring at the very beginning of the scale, where frequencies exceed 10^5 . Beyond this initial cluster, the graph reveals a wide and sparse distribution of much larger sessions. While most interactions involve only a handful of words, rare instances of much higher activity are scattered throughout the spectrum, including a small group of sessions between 5,000 and 10,000 words. The most extreme outlier in the dataset is a single session that reaches nearly 35,000 words, representing an exceptional level of volume compared to the standard user interaction.

The extreme variance within the dataset is formally confirmed through box plot analysis (Figure 7), which provides a clearer perspective on the system’s “edge cases”. While the previous histograms highlight the frequency of short sessions, the Character and Word count distribution in the box plots isolate the sheer distance of outliers from the median.

Furthermore, the Empirical Cumulative Distribution Function (ECDF) (Figure 8) provides a precise quantification of session saturation. Unlike a standard frequency count, the ECDF reveals the specific thresholds where the vast majority of activity is contained.

Building on the session volume analysis, the “Top 30 Most Common Words” (Figure 9) provides a clear window into the actual commands and intent behind the recorded SSH attacks.

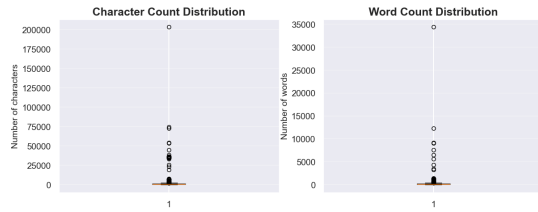


Fig. 7. Box Plots word and character count distribution.

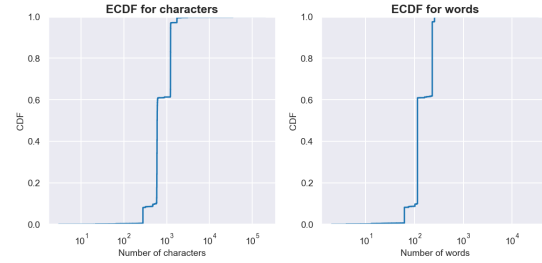


Fig. 8. Empirical Cumulative Distribution Function for characters and words per session.

The data reveals a heavy focus on system reconnaissance and file manipulation, which is characteristic of automated brute-force or post-compromise scripts:

- The most frequent tokens are standard Linux utilities like **grep**, **cat**, and **echo**, with “grep” alone appearing nearly 1.3×10^6 times. These are typically used to search for sensitive information within configuration files or system logs once access is gained.
- There is a significant volume of commands aimed at identifying the target’s hardware and environment, such as **uname**, **/proc/cpuinfo**, and **model**. This suggests attackers are profiling systems to deploy specific, compatible malware.
- The frequent appearance of **rm -rf** indicates attempts to delete logs or system files to cover tracks or cause disruption. Furthermore, the use of **sleep** and hidden directory paths like **.x17-unix** points toward automated scripts designed to remain persistent and evade immediate detection.
- Tokens such as **/var/tmp/.var03522123** and **/tmp/up.txt** highlight the use of temporary directories as staging areas for downloading and executing malicious payloads.

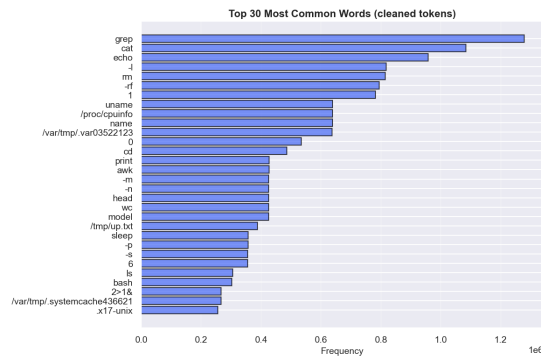


Fig. 9. Top 30 most common words – words over frequencies.

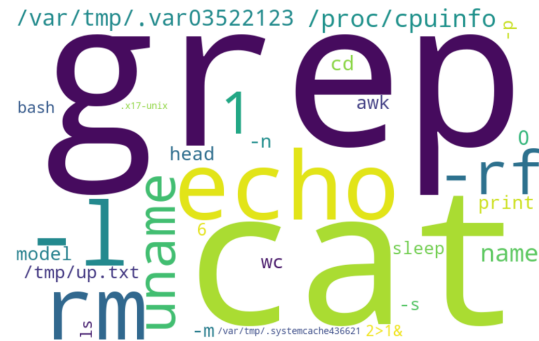


Fig. 10. Word Cloud of the 30 most common words.

The Word Cloud of the 30 most common words (Figure 10) serves as a powerful visual summary that reinforces the frequency data found in the previous histogram.

The analysis of the Distribution of Number of Intents per Session provides a clear view of the complexity of the recorded SSH attacks. The vast majority of attack sessions (Figure 11) are relatively simple, with 2 or 3

intents being the most frequent behaviors, occurring in over 10^5 and 10^4 instances respectively. As the number of intents increases, the frequency drops significantly on a logarithmic scale. Sessions with 4 to 7 intents are rare, representing more sophisticated or multi-stage manual intrusions. Interestingly, sessions with only 1 intent are less common than those with 2 or 3, suggesting that most automated scripts are designed to perform at least a couple of distinct actions.

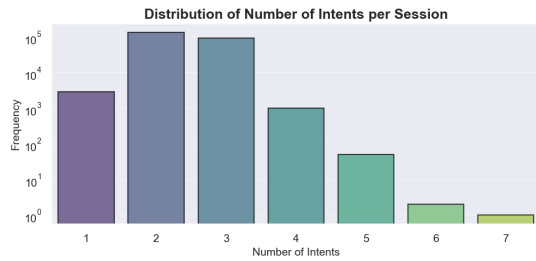


Fig. 11. Distribution of number of Intents per session.

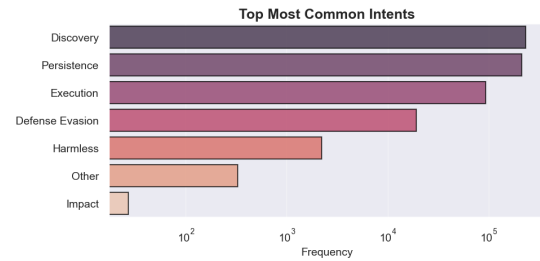


Fig. 12. Top most common intents.

The classification of malicious behavior (Figure 12) into high-level tactical goals reveals a clear hierarchy in the lifecycle of these SSH attacks. The distribution is heavily weighted toward Discovery and Persistence, both of which exceed 10^5 occurrences, indicating that the primary objective of most automated sessions is system profiling and establishing a long-term foothold. This is closely followed by Execution and Defense Evasion, reflecting a systematic progression from reconnaissance to the deployment of malicious scripts while simultaneously attempting to bypass security controls. Conversely, more destructive or high-noise intents like Impact appear with significantly lower frequency, suggesting that attackers prioritize clandestine data gathering and botnet integration over immediate system disruption.

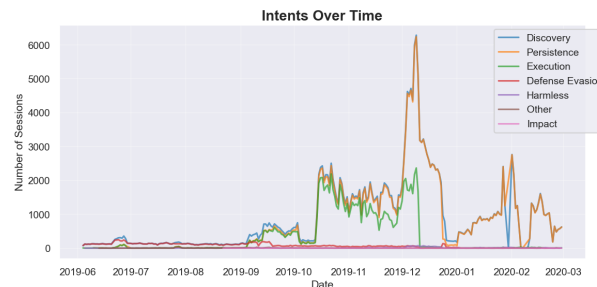


Fig. 13. Intents over time – Number of sessions over Date.

The temporal analysis of attack intents (Figure 13) reveals that the strategic focus of the malicious activity shifted significantly throughout the monitored period. While activity remained at a low, stable baseline for the first several months, a massive surge in Discovery and Persistence intents began in late 2019, peaking in December with over 6,000 sessions per day. This spike was closely mirrored by a rise in Execution intents, suggesting a coordinated campaign that moved rapidly from system profiling to active payload deployment. Following this peak, the volume of discovery-based sessions dropped sharply in early 2020, while persistence-related activity showed a notable secondary resurgence in February, indicating that attackers were likely attempting to maintain or re-establish control over previously compromised systems rather than seeking new targets.

2.2 Pre-Processing: Refining Data Quality

The primary dataset consists of 230,035 unique Unix shell attack instances captured during a honeypot deployment. Each session is categorized based on the specific command sequences executed, and the dataset is structured into five distinct informational columns.

- **session_id**: an integer identifying a session.
- **full_session**: the text related to the full attack session, i.e., the sequence of commands and their parameters (strings).
- **first_timestamp**: when the attack started. (e.g, 2019-06-04 09:45:50.396619+00:00).
- **Set_Fingerprint**: For each attack session, there is a set of intents associated with it. The set is a subset of {'Persistence', 'Discovery', 'Defense Evasion', 'Execution', 'Impact', 'Other', 'Harmless'}.

The “full_session” attribute records raw shell commands executed during each SSH session, serving as a primary source for identifying specific attack vectors. Due to the unstructured nature of these logs, text pre-processing is essential to extract meaningful features and convert them into a numerical format suitable for both supervised and unsupervised learning. During this phase, we identified significant portions of Base64 encoded strings within the sessions. Decoding these revealed hidden Bash commands, suggesting an intentional use of obfuscation to mask malicious activities. A detailed description is provided in Appendix A.

2.3 Pre-Processing: Data transformation

To prepare the dataset for both supervised and unsupervised learning, the raw command logs must be transformed into a structured numerical format. This process involves converting the text into a Bag of Words (BoW) representation, which serves as the foundational step for subsequent TF-IDF (Term Frequency-Inverse Document Frequency) weighting

2.3.1 Bag of Words (BoW). The initial transformation converts qualitative text into a mathematical matrix where the frequency of each command is recorded. A specialized `CountVectorizer` is employed using a non-whitespace token pattern (`r'\S+'`) to ensure that technical artifacts, such as directory paths and command flags, are preserved as single entities.

The feature space is constrained to the top 1,000 most frequent tokens and filtered through a custom list of “stop words” to eliminate non-informative session noise, focusing the model on high-signal behavioral indicators.

The resulting BoW matrix represents each attack session as a vector, providing a quantitative basis for analyzing the tactical surges and shifts in intent observed throughout the monitoring period.

2.3.2 Term Frequency-Inverse Document Frequency (TF-IDF). To refine the numerical representation for machine learning, the TF-IDF (Term Frequency-Inverse Document Frequency) algorithm was applied to the pre-processed session data. While the previous Bag of Words (BoW) approach simply counted occurrences, TF-IDF weighs tokens based on their informational value, effectively distinguishing between routine system noise and unique attack signatures.

The TF-IDF transformation converts the cleaned sessions into a weighted matrix by calculating a score for each term t in a document (session) d within the total dataset D . The score is derived from the product of two distinct statistics:

- Term Frequency ($TF(t, d)$): this measures how frequently a command (t) appears in a single session (d).

$$TF(\text{word}, \text{session}) = \frac{\text{Number of times the word appears in the session}}{\text{Total number of words in that session}} \quad (1)$$

- Inverse Document Frequency ($IDF(t, D)$): This measures how rare a term is across the entire dataset, calculated as:

$$IDF(\text{word}, \text{session}) = \log \left(\frac{\text{Total number of sessions } N}{1 + \text{Number of sessions containing the word}} \right) \quad (2)$$

The final weight is the product of these two values ($TF * IDF$). This methodology focuses the analysis on the most tactically significant behaviors observed during the monitoring period. By calculating the IDF, generic commands that appear in almost every session receive a score near zero. This prevents automated "noise" from skewing the results of the supervised learning models. Unique strings associated with Persistent or Execution, such as specific hidden directory names or encoded payloads, receive the highest weights due to their rarity across the broader dataset. The resulting weighted matrix is serialized and stored, allowing these importance scores to be reused across different supervised and unsupervised tasks. This ensures that the time-consuming computation of importance weights does not need to be repeated during model training.

3 Supervised Learning

In this chapter, various models for classifying attack session tactics are trained. Each session can have multiple labels. Initially, the dataset is split to facilitate a training phase using default parameters, after which performance is evaluated. Following this, hyperparameters are tuned to train other models to observe performance changes. Finally, results are discussed, and different feature combinations are tested. To simplify the comparison of model performance, all results and metrics are stored in a Pandas DataFrame and exported to an **Excel file**.

3.1 Dataset split

This section aims to prepare the dataset for training session. The dataset is partitioned into training and testing sets. Before splitting the data, the lists of fingerprints are transformed using Scikit-learn's `MultiLabelBinarizer` into a binary matrix. Each column represents a label (one of the seven attack classes), and each row indicates the presence (1) or absence (0) of a specific tactic within the attack session.

The process begins by loading the `X_tfidf` feature matrix, generated in the previous section, which consists of 233,035 samples and 1000 features. Subsequently, the Scikit-learn function `train_test_split` is utilized to randomly partition both the feature matrix and the target label matrix into a training set (80%) and a test set (20%). Consequently, 186,428 samples are used to train the model and 46,607 are reserved to test it on new attack sessions.

3.2 Training of two models

The Random Forest Classifier was selected as the first model due to its robustness and capability to handle **multi-label classification** tasks effectively in high-dimensional feature spaces. The model was trained using the feature matrix defined in subsection 3.1. The training process required approximately 7 minutes.

Regarding performance evaluation, the accuracy was computed by comparing predicted labels against ground truth labels for both the training and test sets. The model achieved an accuracy of 99% on the training set and 98% on the test set.

However, the classification report (Table 1) on the test set reveals varying performance depending on the attack tactic. The most represented classes, such as *Discovery*, *Execution*, and *Persistence*, exhibit excellent scores. Conversely, minority classes present significant challenges. The 'Harmless' class is particularly problematic: while precision is high (92%), recall is critically low (18%). This indicates that 82% of actual 'Harmless' instances are misclassified (False Negatives). Finally, metrics for the 'Impact' class (only 2 samples) are not statistically significant.

Table 1. Classification Report (Test) - Random Forest

Class	Precision	Recall	F1-Score	Support
Defense Evasion	1.00	0.97	0.98	3,760
Discovery	1.00	1.00	1.00	46,433
Execution	1.00	0.99	0.99	18,512
Harmless	0.92	0.18	0.31	458
Impact	1.00	0.50	0.67	2
Other	1.00	0.99	0.99	67
Persistence	1.00	1.00	1.00	42,281
<i>micro avg</i>	1.00	0.99	1.00	111,513
<i>macro avg</i>	0.99	0.80	0.85	111,513
<i>weighted avg</i>	1.00	0.99	1.00	111,513

To visually analyze the misclassifications, the multilabel confusion matrices were generated using the `plot_mcm` function (Figure 14).

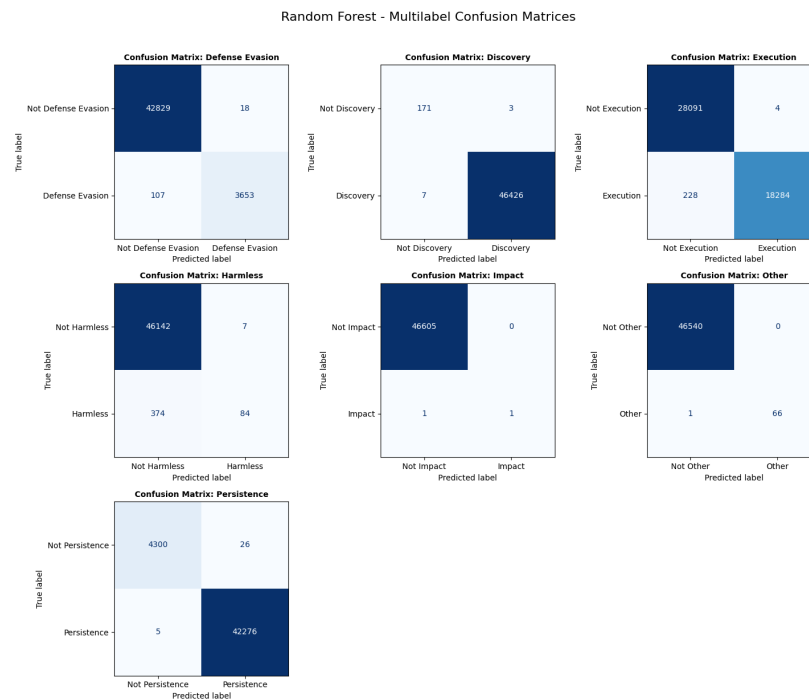


Fig. 14. Random Forest confusion matrices for each label

The minimal difference between training (99%) and test (98%) accuracy suggests the absence of significant overfitting; the model generalizes well on majority classes. The primary limitation remains **class imbalance**.

Since the Random Forest algorithm tends to prioritize majority classes to maximize global accuracy, this issue motivates the strategies applied in the next section regarding Hyperparameter Tuning and Cross-Validation.

The **Logistic Regression** (wrapped in a **MultiOutputClassifier**, because this strategy transforms the complex multi-label problem into many simple binary problems that Logistic Regression can solve) was chosen as the second model because it is particularly effective for high-dimensional sparse data, such as TF-IDF vectors. The model was trained using the same feature matrix. Notably, the training process was significantly faster, requiring approximately 1 minute.

Regarding metrics, the model achieved an accuracy of 98% on both training and test sets, showing no signs of overfitting. However, the classification report (Table 2) reveals results similar to Random Forest. While represented classes have excellent scores, minority classes suffer. The 'Harmless' class shows high precision (98%) but very low recall (13%), meaning 87% of actual 'Harmless' instances are missed.

Table 2. Classification Report (Test) - Logistic Regression

Class	Precision	Recall	F1-Score	Support
Defense Evasion	0.99	0.97	0.98	3,760
Discovery	1.00	1.00	1.00	46,433
Execution	1.00	0.99	0.99	18,512
Harmless	0.98	0.13	0.23	458
Impact	0.00	0.00	0.00	2
Other	1.00	0.99	0.99	67
Persistence	1.00	1.00	1.00	42,281
<i>micro avg</i>	1.00	0.99	1.00	111,513
<i>macro avg</i>	0.85	0.72	0.74	111,513
<i>weighted avg</i>	1.00	0.99	0.99	111,513

In conclusion, Both models exhibit comparable predictive performance, but they share the same weakness regarding minority classes. However, Logistic Regression proves to be computationally more efficient, being approximately seven times faster than Random Forest.

Finally, regarding generalization, the analysis reveals **no significant overfitting** for either model, as evidenced by the negligible gap between training and test accuracies ($< 1\%$). This stability confirms that both algorithms generalize well to unseen data, although their high aggregated performance is primarily driven by the dominant attack classes.

3.3 Hyper-parameters Tuning

To do this optimization in terms of the classification accuracy and analyzing model stability, a hyper-parameter tuning step using GridSearchCV was conducted. The main objective of this step is to check if optimized hyper-parameters can improve minority class identification (*Harmless* and *Impact*) over default values used in the baseline model. To avoid high computation time during model grid search, a subsampling approach with a 3-fold Cross-Validation technique on a reduced set of 15,000 samples using Grid Search was used. Model parameters were optimized based on Weighted F1-score; hence, optimized models were re-fit on the entire dataset and evaluated using a separate Test Set.

For Random Forest Classification, hyper-parameter tuning emphasized a balance between model complexity and a good level of generalization capacity. Hyper-parameters were optimized using a grid search with the following combinations:

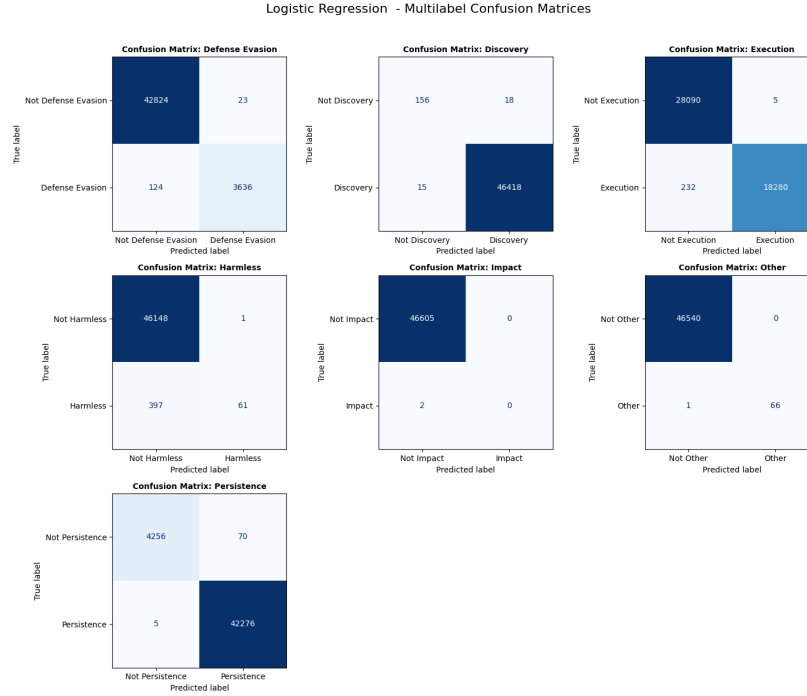


Fig. 15. Logistic Regression Multilabel confusion matrices

- **n_estimators** {30, 50, 100}: To check if a smaller number of trees can deliver competitive performance.
- **max_depth** {10, 20, None}: To control overfitting.
- **min_samples_leaf** {1, 2}: To make a margin among decisions. Hyper-parameter tuning produced results with maximum complexity parameters ($n_estimators = 100$, $max_depth = None$, $min_samples_leaf = 1$) indicated that a deep model using deep trees can better grasp TF-IDF space intricacies.

For MultiOutput Logistic Regression, tuning emphasized regularization and class imbalance modeling:

- **C** {0.1, 1, 10}: Inverse of regularization strength.
- **class_weight** {None, 'balanced'}: To provide automatic class weights based on frequency.

The outcomes of the Grid Search are visualized in Figure 16 and Figure 17. The heatmaps display the Weighted F1-Score across different hyperparameter combinations.

The two plots have a small structural difference. The Random Forest heatmap (left) comprises three rows, corresponding to the three tested values for max_depth (where 'nan' represents None, 10, and 20). Conversely, the Logistic Regression heatmap (right) features one fewer row. This is strictly due to the grid definition for the $class_weight$ parameter, which compared only two configurations None and balanced.

The optimized hyper-parameters were obtained: $C = 10$, indicating a need to fit models very closely despite regularization, and $class_weight = None$.

3.3.1 Performance Comparison and Analysis. The results highlight a strong robustness of the baseline configurations (Table 3). The comparison shows that the optimized models do not significantly outperform the baseline ones, indicating that the original hyperparameters were already well suited for this specific task. This behavior

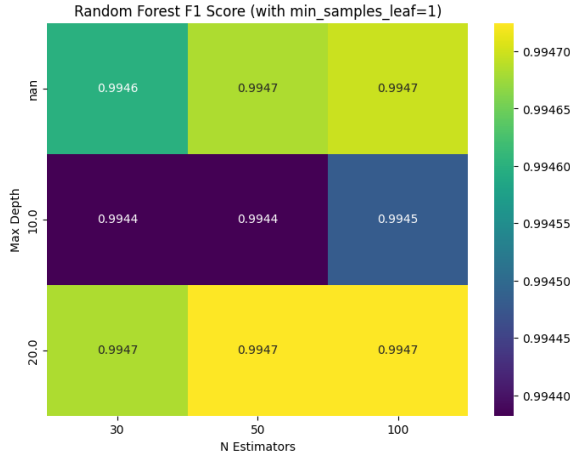


Fig. 16. Random Forest Grid Search F1 Score (Weighted) varying depth and estimators.

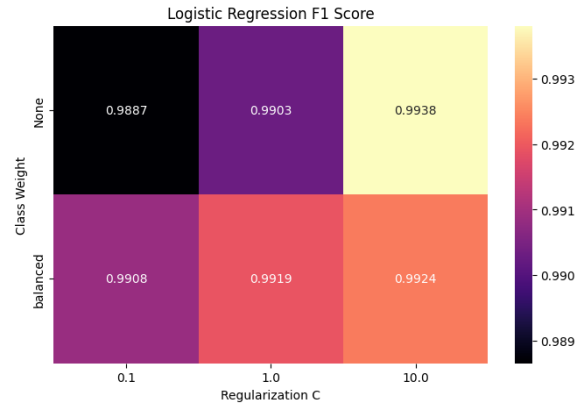


Fig. 17. Logistic Regression Grid Search F1 Score (Weighted) varying C and Class Weight.

can be attributed to the distinctive nature of the command-based features, which allow the models to achieve near-perfect accuracy for most intent classes, such as *Discovery* and *Persistence*, regardless of further parameter tuning.

Table 3. Comparison of Best Hyper-parameters and Performance Metrics

Model	Best Hyper-parameters Found	Val. F1 (W)	Test F1 (W)	Test F1 (M)
Random Forest	n_estimator: 100 max_depth: None min_samples_leaf: 1	0.9947	1.00	0.85
Logistic Regression	C: 10 class_weight: None	0.9939	1.00	0.75

Another important aspect is the strong consistency between validation and test performance. The Validation F1-scores of 0.9947 for the Random Forest model and 0.9939 for Logistic Regression are closely aligned with the Test Weighted F1-score of 1.00. This consistency suggests that both models generalize well to unseen data and do not suffer from overfitting, reinforcing their reliability in real-world scenarios.

However, a noticeable discrepancy emerges when comparing Weighted F1 and Macro F1 scores. While the Weighted F1 reaches a perfect score of 1.00, the Macro F1 drops to 0.85 for Random Forest and 0.75 for Logistic Regression. This gap is primarily caused by the extremely low representation of *Impact* and *Harmless* samples in the dataset. Although the models classify most traffic samples with almost perfect accuracy, they struggle to achieve complete recall on these minority classes, which significantly affects the Macro F1 metric.

Finally, even after hyperparameter tuning, Random Forest maintains a clear advantage over Logistic Regression, particularly in terms of Macro F1-score. This result further confirms the superior ability of Random Forest to handle the complexity and multi-label characteristics of SSH session data, making it a more effective choice for this classification task.

3.4 Result for each intent

A detailed intent-level analysis reveals a clear distinction between highly prevalent classes and minority classes. The most frequent intents are identified with near-perfect accuracy by both classifiers, while performance degrades noticeably for underrepresented classes, where severe class imbalance becomes the dominant limiting factor for correct classification.

The intents *Discovery* and *Persistence* represent the largest portion of the dataset and are classified with perfect performance by both models, achieving an F1-score of 1.00 with precision and recall equal to 1.00. This result suggests that the command patterns associated with these intents, such as scanning activities or standard download and persistence mechanisms, are highly distinctive. In the TF-IDF feature space, these patterns appear to be well separated, allowing even linear decision boundaries to discriminate them effectively.

Similarly, *Execution* and *Defense Evasion* also exhibit very strong performance, with recall values greater than or equal to 0.97. The slight reduction in recall observed for Defense Evasion can be attributed to a small number of false negatives. These misclassifications are likely caused by obfuscated commands or operations that appear benign in isolation or overlap with other attack phases, making them more challenging to identify with absolute certainty.

Interestingly, the *Other* intent, despite being a broad and somewhat heterogeneous category, achieves a high level of accuracy, with an F1-score close to 0.99. This indicates that sessions grouped under this label are not entirely random but instead share structural similarities or recurring keywords that differentiate them from the more clearly defined attack-related intents.

In contrast, the *Harmless* and *Impact* intents clearly highlight the limitations of supervised learning under extreme class imbalance conditions. With only 458 and 2 samples respectively in the test set, these classes are particularly challenging to model. For Harmless sessions, both classifiers exhibit a conservative prediction strategy characterized by high precision (approximately 0.93) but very low recall (around 0.18). This behavior indicates that when a session is predicted as benign, the prediction is usually correct; however, most benign sessions are either misclassified or absorbed into malicious categories, reflecting a strong bias toward avoiding false negatives at the cost of missing true positives.

In particular, the *Impact* intent represents an extreme edge case. With only two available test samples, Logistic Regression fails entirely to identify this class, resulting in a recall of 0.00. Random Forest, on the other hand, can correctly identify half of the samples, achieving a recall of 0.50. Although this result is still far from optimal, it demonstrates a clear advantage of ensemble-based approaches. The ability of Random Forest to isolate small sample regions within its decision trees allows it to capture rare patterns that linear models, such as Logistic Regression, tend to dismiss as noise.

Overall, these findings confirm that while both models are highly effective in identifying dominant attack intents, Random Forest shows superior robustness when dealing with rare and difficult classes, reinforcing its suitability for complex and imbalanced security datasets.

3.5 Feature Exploration and Ablation Study

To better understand which factors influence the model's performance and to assess whether a more complex TF-IDF representation is necessary, an ablation study was conducted. Starting from the full model, two simplified feature configurations were evaluated: a Bag of Words representation, which relies solely on raw term frequencies without IDF weighting, and a Metadata Only configuration, which completely ignores command text and uses only two scalar features per session, namely the total number of characters and the total number of words.

The outcome of this comparative analysis can be viewed in Table 4. The comparison between TF-IDF and Bag of Words reveals an almost complete equivalence in performance. In the case of Random Forest, both representations achieve the same F1-score (0.9965), indicating that the addition of IDF weighting does not provide any measurable

Table 4. Comparison of Model Performance (Weighted F1-Score) across different Feature Sets

Feature Set	Random Forest	Logistic Regression
TF-IDF (Baseline)	0.9965	0.9960
Bag of Words (BoW)	0.9965	0.9964
Metadata Only	0.9961	0.9894

advantage. For Logistic Regression, Bag of Words performs marginally better, with an improvement of 0.0004. This slight difference can be attributed to the IDF penalty applied to very common terms such as “grep” or “cd”, which, although frequent, are strong indicators of the majority class (*Discovery*) in this dataset. Penalizing them therefore slightly reduces their usefulness. Overall, these results suggest that the relative weighting of terms plays a minor role, while the mere presence or absence of key commands is sufficient to discriminate between attack intents. The attack patterns are so distinct that a simple frequency-based representation already captures the necessary information for near-perfect classification.

Even more surprising is the performance achieved by the Metadata Only configuration. Using only session length measured in characters and words, the Random Forest model reaches an F1-score of 0.9961, which is virtually indistinguishable from the text-based approaches. This result indicates a strong correlation between the structure of a session and its underlying intent.

The fact that such minimal metadata can yield such high accuracy suggests that the dataset contains highly repetitive and automated attack behaviors. In this context, the “shape” of the traffic, namely session length, is almost as distinctive as the command content itself. However, relying solely on metadata would be problematic in a real-world scenario, as an adaptive attacker could easily manipulate session length by padding commands without changing the actual payload, thereby evading detection.

Although TF-IDF was ultimately selected as a robust and content-aware baseline, this ablation study demonstrates that, for this specific dataset, the predictive information is largely redundant. The exceptionally strong performance achieved using only basic session statistics indicates that the attacks follow predictable templates and are executed in a fully automated manner, making them trivial to classify regardless of the sophistication of the feature representation.

4 Unsupervised Learning

Before applying clustering algorithms to uncover hidden patterns within SSH sessions, an additional preprocessing step was required to address the high dimensionality and sparsity of the TF-IDF feature space, which originally consisted of 233,035 samples and 1,000 features. High-dimensional spaces are well known to negatively affect the efficiency and reliability of distance-based algorithms such as K-Means. This phenomenon, commonly referred to as the *Curse of Dimensionality*, causes Euclidean distances to lose discriminative power as dimensionality increases, making clustering results less meaningful.

To mitigate this issue, a dimensionality reduction strategy was adopted. Instead of applying standard Principal Component Analysis (PCA), which would be computationally expensive and inefficient given the sparse nature of TF-IDF representations, Singular Value Decomposition (SVD) was employed. SVD is particularly well suited for sparse text-based data and allows for efficient projection into a lower-dimensional space. Through this approach, the dimensionality was reduced from 1,000 to 100 components, achieving a cumulative explained variance of 82.66%. This result indicates that a substantial portion of the original variance, and thus the underlying structure of the attack commands, was preserved, while a large amount of noise and non-informative variation was effectively removed.

Finally, although clustering is inherently an unsupervised learning task and does not require labeled data, ground truth labels were prepared for evaluation purposes. These labels were derived from the fingerprint column using a MultiLabel Binarizer and were intentionally excluded from the clustering process itself. They were reserved exclusively for external evaluation which will be presented in Section 3.5. This separation ensures an objective assessment of clustering quality and enables a meaningful comparison with known MITRE ATT&CK tactics through external validation metrics such as Purity and the Adjusted Rand Index (ARI).

4.1 Determining the Optimal Number of Clusters

To analyze the underlying structure of the unlabeled dataset and determine the optimal number of clusters (k), an iterative analysis using K-Means was performed on the reduced feature space obtained via SVD. The analysis explored values of k in the range $[2, 15]$ and employed two complementary assessment methods:

- **Clustering Error (Inertia):** This metric evaluates cluster compactness by calculating the Sum of Squared Errors (SSE) between data points and their respective centroids. The goal is to identify the “elbow point,” beyond which increasing k provides only marginal improvements in cluster compactness.
- **Silhouette Coefficient:** This metric assesses clustering quality by comparing intra-cluster cohesion to inter-cluster separation. Higher values indicate better-defined, more clearly separated clusters.

The upper bound of the k range was set to 15 based on heuristic reasoning derived from the available Ground Truth. The dataset contains 7 known tactics from the MITRE ATT&CK framework, which serve as an initial estimate for the number of high-level behavioral groups. To allow the algorithm to explore potential sub-structures within these tactics (e.g., different operational patterns within the Discovery tactic), the upper limit was doubled to 14 and slightly extended to 15. This ensures sufficient flexibility for detecting sub-clusters while avoiding over-segmentation, which could lead to noisy or poorly defined clusters.

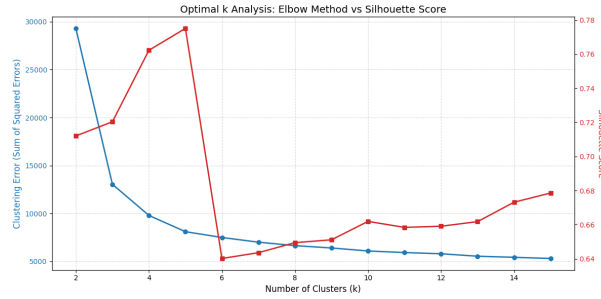


Fig. 18. Determination of the optimal number of clusters (k) using Elbow Method and Silhouette Score.

The results, summarized in Figure 18, show the following trends:

- **Clustering Error (Inertia):** The inertia curve exhibits a steep initial decline, with a clear inflection point between $k = 4$ and $k = 5$. Beyond $k = 5$, the decrease in error becomes marginal, indicating diminishing returns from adding additional clusters.
- **Silhouette Score:** The silhouette score provides a more decisive indication. It increases gradually and reaches a global maximum of 0.7751 at $k = 5$. Increasing k to 6 causes a sharp drop (approximately 0.64), suggesting that adding another cluster disrupts the cohesion of already well-formed clusters.

Based on this evidence, $k = 5$ was selected as the optimal number of clusters. While the Elbow method suggests that a plausible range could be between 4 and 5, the silhouette analysis clearly identifies $k = 5$ as the point of

maximal cluster separation and cohesion. Values beyond this threshold are likely to introduce noise without meaningful gains in clustering quality.

4.2 Hyper-parameter Tuning

After identifying the optimal number of clusters (k=5), the next step focused on fine-tuning the internal parameters of the K-Means algorithm. The goal was to verify whether the clustering results corresponded to a global minimum and whether the clusters had reached a stable configuration. A Grid Search was performed, targeting three key parameters:

- **Initialization (init):** Comparison between the traditional k-means++ initialization, which spreads centroids strategically, and random initialization.
- **Number of Restarts (n_init):** Evaluation of 10 versus 20 restarts, to avoid convergence to a local minimum.
- **Algorithm:** Comparison between the standard lloyd algorithm and elkan, which can be more computationally efficient for dense data but may consume more memory.

Table 5. K-Means Clustering Performance Comparison with different Initialization Strategies

Initialization	n_init	Algorithm	Silhouette Score	Clustering Error (Inertia)
k-means++	10	lloyd	0.7751	8101.8
k-means++	20	lloyd	0.7751	8101.8
random	20	lloyd	0.6213	9337.37
k-means++	10	elkan	0.7751	8101.8

The tuning process was conducted on a validation subset of 15.000 samples, balancing computational efficiency with reliability. The results are summarized in Table 5.

A clear performance degradation is observed when using random initialization, despite an increased number of restarts. Compared to k-means++, random initialization results in a significantly lower Silhouette Score and higher clustering error, indicating convergence to a sub-optimal local minimum.

This behavior highlights the presence of local minimum in the clustering landscape and demonstrates that convergence quality is strongly influenced by centroid initialization. In contrast, all configurations using k-means++ consistently converge to the same optimal solution, regardless of the number of restarts or the algorithm variant. This confirms the robustness of the clustering structure in the reduced SVD feature space when appropriate initialization is employed.

The results also show that increasing n_init from 10 to 20 does not provide additional benefits when using k-means++, suggesting that convergence to the global minimum is achieved reliably within 10 restarts.

Based on these findings, the final configuration was automatically selected as the best-performing one according to the Silhouette Score maximization criterion applied during the Grid Search.

Among all tested configurations, the combination init='k-means++', n_init=10, and algorithm='lloyd' consistently achieved the highest Silhouette Score while minimizing clustering error. Configurations relying on random initialization converged to sub-optimal local minima, despite an increased number of restarts.

The selection of this configuration can be interpreted as follows:

- init='k-means++' was selected because it empirically prevents convergence to sub-optimal local minimum and ensures stable clustering outcomes.
- n_init=10 was sufficient to reliably reach the optimal solution; increasing the number of restarts did not provide measurable improvements.

- `algorithm='lloyd'` achieved the same clustering quality as `elkan`, while offering lower memory usage and better suitability for the reduced-dimensional feature space.

Overall, this configuration represents the optimal trade-off between clustering quality, stability, and computational efficiency as determined by the hyper-parameter search procedure. To validate the effectiveness of the hyper-parameter optimization, the performance of the tuned model was compared against the baseline configuration used in subsection 4.1.

Table 6. Performance Comparison between Baseline and Tuned K-Means Models

Model Version	Configuration	Inertia	Silhouette Score
Baseline	<code>n_init=10, init=k-means++</code>	8102.0	0.7751
Tuned	<code>n_init=10, init=k-means++</code>	8101.8	0.7751

The results in Table 6 show that the baseline and tuned models achieve effectively identical performance. This indicates that the baseline configuration had already converged to the optimal clustering solution.

However, the hyper-parameter tuning process remains essential, as it demonstrates that this optimal solution is **not** guaranteed across all parameter configurations. Alternative settings relying on random initialization were shown to converge to sub-optimal local minima, resulting in significantly lower Silhouette Scores and higher clustering error.

Therefore, the tuning phase serves to validate the optimality and robustness of the baseline configuration, rather than to improve it numerically. The consistency between the baseline and tuned results confirms that the five clusters identified represent the intrinsic structure of the attack patterns when appropriate initialization is employed.

4.3 Visualization of Clusters

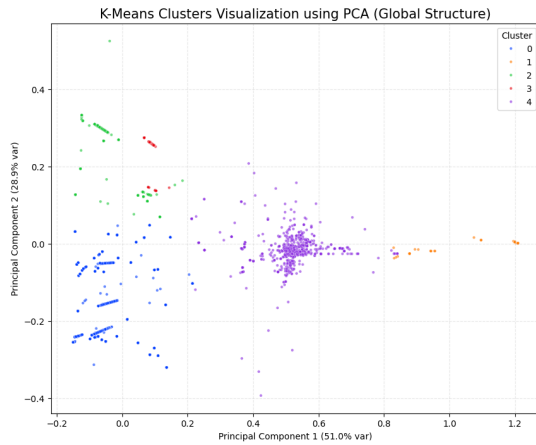
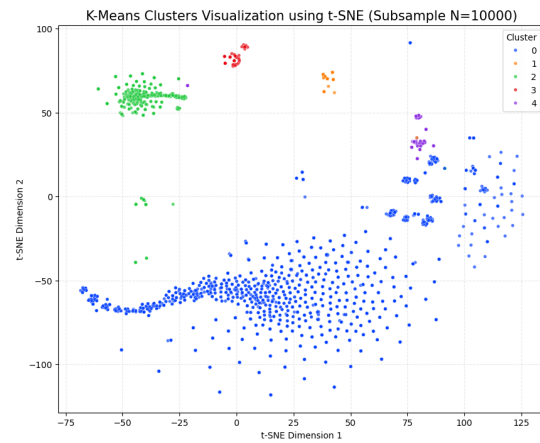
To gain an intuitive geometric understanding of the discovered clusters and to further justify the choice of $k = 5$, two complementary dimensionality reduction techniques were employed to project the 100-dimensional feature space into two dimensions. Each method provides a different perspective on the cluster structure.

4.3.1 Global Structure Analysis (PCA). Principal Component Analysis (PCA) was first applied to visualize the global variance structure of the dataset. The resulting projection is highly informative: the first two principal components account for approximately 79.9% of the total variance (PC1: 51.0%, PC2: 28.9%), indicating that the data is well represented in a two-dimensional space.

The PCA visualization reveals that the data points do not collapse into a single amorphous cluster. Instead, a clear radial structure emerges, with multiple elongated branches extending from a central region. These branches correspond to distinct behavioral patterns and are consistently identified by the K-Means algorithm as separate clusters. **Clusters 2 and 3** (highlighted in green and red, respectively) occupy clearly separated directions in the PCA space, suggesting substantial differences in their underlying command composition or operational characteristics. This global separation supports the existence of a limited number of well-defined high-level groups (Figure 19).

4.3.2 Local Manifold Analysis (t-SNE). While PCA emphasizes global variance, it may obscure fine-grained local relationships. To complement this view, t-Distributed Stochastic Neighbor Embedding (t-SNE) was applied to a representative subsample of 10,000 data points to preserve local neighborhood structure.

The t-SNE visualization effectively unfolds the high-dimensional manifold and provides strong confirmation of the clustering results. The five clusters appear as compact, self-contained regions separated by clear empty

Fig. 19. PCA projection of K-Means clusters ($k = 5$).Fig. 20. t-SNE visualization of K-Means clusters ($k = 5$).

margins, indicating strong intra-cluster cohesion and inter-cluster separation. This observation aligns closely with the high Silhouette Score obtained during quantitative evaluation.

Notably, **clusters 2 (green) and 1 (orange)** form particularly tight and isolated regions, suggesting highly repetitive and specific attack behaviors. In contrast, **cluster 0 (blue)** exhibits a much more diffuse and scattered structure across the lower manifold, likely acting as a heterogeneous or “background” cluster that aggregates more diverse or less structured attack sessions (Figure 20).

4.4 Semantic Characterization of Clusters

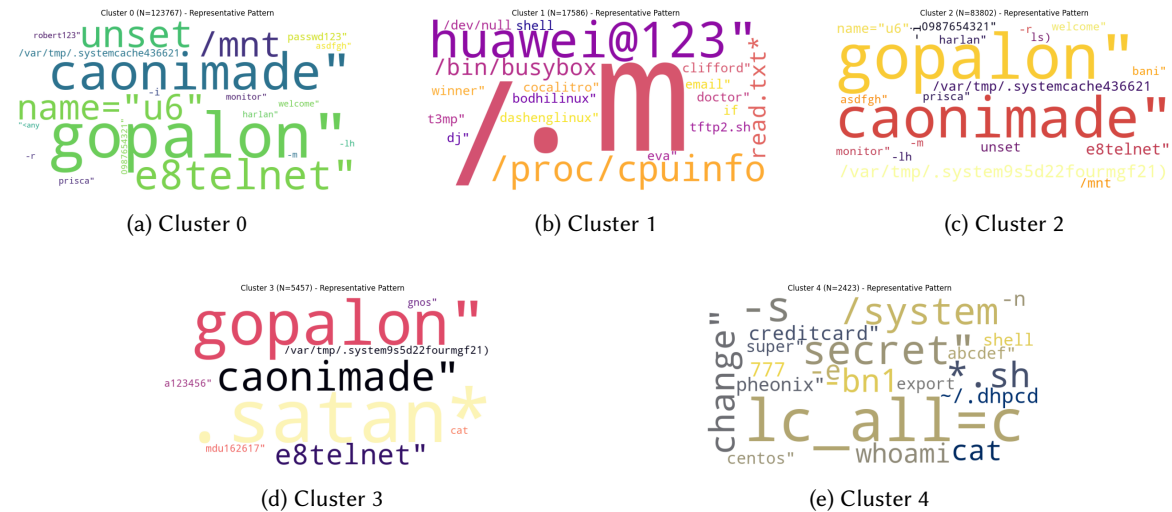


Fig. 21. Top frequent terms and commands for each K-Means cluster.

To interpret the clustering results in a statistically grounded yet behaviorally meaningful way, we performed a semantic analysis of the five clusters generated by the unsupervised model. This analysis was based on the inspection of cluster centroids and the most frequently occurring commands within each group. As illustrated in Figure 21, Word Cloud visualizations were generated for each cluster to highlight the most dominant terms and command patterns.

The sessions grouped in Cluster 0 exhibit a highly consistent and repetitive command vocabulary, strongly centered around keywords such as `gopalon`, `e8telnet`, and `caonimade`. These terms are widely recognized as identifiers commonly associated with large-scale attacks targeting IoT and embedded devices. Their frequent co-occurrence with environment manipulation commands, such as variable unsetting and parameter assignments, alongside references to writable directories like `/mnt` and `/var/tmp/`, suggests the execution of automated workflows. The repeated appearance of structured file paths (e.g., `/var/tmp/.systemcache436621`) and uniform execution arguments further reinforces the interpretation of Cluster 0 as representing scripted, non-interactive system interactions performed at scale.

A partially overlapping but distinct behavior emerges in Cluster 2, which shares many lexical features with Cluster 0, including the recurring presence of `gopalon`, `caonimade`, and `e8telnet`. Despite this similarity, Cluster 2 displays consistent variations in temporary file naming conventions and payload placement paths, such as alternative hidden directory structures under `/var/tmp/`. These differences, although subtle, are systematic and suggest the use of parallel execution templates or closely related malware variants. The strong vocabulary overlap indicates a shared operational ecosystem, while the divergence in implementation details points to distinct infection routines rather than random noise.

In contrast to the highly automated patterns observed in Clusters 0 and 2, Cluster 1 is characterized by more selective and information-driven command sequences. Sessions within this group frequently include explicit authentication attempts using vendor-associated credential strings, such as `huawei@123`, alongside architecture inspection commands like `cat /proc/cpuinfo`. The repeated invocation of `/bin/busybox` highlights an effort to maintain compatibility across heterogeneous systems, while file enumeration and conditional execution patterns indicate that actions are adjusted dynamically based on the characteristics of the compromised host. Collectively, these features reflect an interaction model that prioritizes system fingerprinting and contextual awareness over blind execution.

A markedly different operational profile is observed in Cluster 3, which is distinguished by the presence of long, densely encoded command strings, often employing Base64 obfuscation. These payloads are typically embedded within shell execution contexts and are designed to conceal their functionality from signature-based detection mechanisms. The recurrent appearance of markers such as `.satan`, combined with commands suggesting decoding, unpacking, and execution of externally retrieved content, indicates a higher level of technical sophistication. Compared to other clusters, the emphasis here is not on enumeration or environment probing but on stealth, obfuscation, and controlled execution.

Finally, Cluster 4 aggregates sessions dominated by generic system interaction commands and environment configuration parameters. Frequently observed elements include locale normalization settings (e.g., `lc_all=c`), identity queries such as `whoami`, file inspection commands like `cat`, and generic script execution patterns involving wildcarded shell scripts. Unlike the other clusters, Cluster 4 lacks a small set of distinctive signature keywords. Instead, it is defined by the co-occurrence of widely used system utilities and flexible command chaining. This absence of strong lexical markers results in a more heterogeneous composition, making it difficult to infer intent from isolated commands alone.

4.5 External Evaluation: Homogeneity and Intent Division

To evaluate the clustering results, the five clusters, described previously, were compared against the MITRE ATT&CK Ground Truth labels, which categorize attack sessions into high-level tactical groups. Two complementary metrics were employed: Purity, measuring the extent to which a cluster contains sessions from a single Ground Truth label, and the Adjusted Rand Index (ARI), which quantifies overall agreement between the clusters and Ground Truth while accounting for chance.

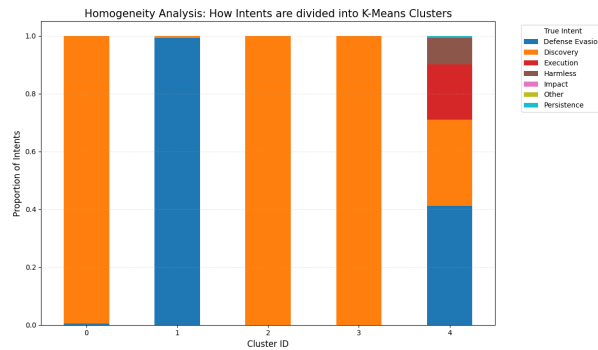


Fig. 22. Distribution of MITRE ATT&CK Ground Truth labels across K-Means clusters.

As illustrated in Figure 22, the analysis reveals a noteworthy pattern. The overall Purity score is very high (0.9910), indicating that most clusters are internally homogeneous with respect to the dominant Ground Truth label. In contrast, the ARI is considerably lower (0.2238), suggesting that the clustering does not simply replicate the Ground Truth taxonomy. This discrepancy indicates that while clusters generally contain sessions from a single label, the algorithm distinguishes subgroups within the same Ground Truth category based on intrinsic patterns in the data.

Sessions labeled as *Discovery* in the Ground Truth are distributed across Clusters 0, 2, and 3. Although they share the same high-level label, each cluster represents a distinct group in the unsupervised clustering, as reflected in their separate cluster assignments. Cluster 1 is almost entirely composed of sessions labeled as *Defense Evasion* (99.27%), showing a strong correspondence between this cluster and the Ground Truth label. Cluster 4 displays the lowest dominance of any single Ground Truth label, with 41.27% of sessions associated with *Defense Evasion*, indicating that this cluster is more heterogeneous compared to the others.

These metrics demonstrate that the unsupervised clustering preserves homogeneity within clusters while also differentiating between subgroups that share the same high-level tactical category.

4.6 Fine-Grained Attack Categorization

To uncover specific attack campaigns and toolsets, this section integrates quantitative N-gram sequence mining and Co-occurrence Graphs with a qualitative manual inspection of raw logs. This multi-layered approach isolates distinct operational behaviors, revealing sophisticated patterns that extend beyond broad tactical classifications.

Cluster 4 acts not as a heterogeneous error, but as a cohesive representation of a full Kill Chain attack cycle. The Cooccurrence Graph Figure 23 visualizes a robust semantic link between file transfer utilities and execution binaries, confirming this cluster represents the "Staging Phase" of an infection.

This is supported by N-gram analysis, which highlights a repetitive sequence of events: navigating to writable directories (`cd /tmp`), modifying permissions (`chmod 777`), and clearing environment variables (`lc_all=c`). Crucially, the manual inspection of raw logs revealed what statistical models missed: the extensive use of



Fig. 23. Co-occurrence network of Cluster 4.



Fig. 24. Co-occurrence network of Cluster 0.

command chaining (e.g., `cd /tmp && wget ... && chmod +x`). Attackers use these single-line commands to automate the download-and-execute process, utilizing tools like `wget` to fetch payloads from remote servers.

The analysis of Cluster 0 provides the most significant insight. Although statistically dominated by *Discovery* labels, the fine-grained analysis exposes a sophisticated Resource Hijacking campaign.

The Co-occurrence Graph (Figure 24) links reconnaissance commands directly to system modification tools. While the most frequent N-grams appear harmless, focusing on hardware enumeration like `cat /proc/cpuinfo` or `grep`, the manual review discloses the true malicious logic.

These hardware scans are essentially "resource validation" steps immediately followed by a Competitive Account Takeover. The raw logs show syntax such as `echo -e '...' | passwd`, demonstrating that the malware validates the device's CPU power and then attempts to change the root password. This behavior is indicative of botnet turf wars: the attacker secures exclusive access to the device by locking out the legitimate owner and preventing rival botnets from exploiting the same host.

5 Language Models Exploration (BERT)

5.1 Introduction and Methodology

In this section, we move beyond traditional statistical embeddings to explore Large Language Models (LLMs), specifically applying BERT to the task of SSH command classification.

BERT represents a significant advancement in NLP due to its bidirectional nature. Unlike previous directional models, BERT utilizes the Transformer encoder architecture to learn the context of a word based on both its previous and next surroundings simultaneously. This capability is learned during pre-training via two unsupervised tasks: Masked Language Modeling (MLM) and Next Sentence Prediction (NSP).

5.2 Implementation Strategy: Transfer Learning

Since SSH commands exhibit complex syntactical structures similar to natural language, we hypothesize that BERT can capture contextual nuances that simpler models (like TF-IDF) might miss.

To implement this, we utilized Transfer Learning. We adopted the pre-trained bert-base-uncased model to serve as a high-level feature extractor. By freezing the internal weights of the BERT backbone, we retained its pre-learned linguistic knowledge and trained only a custom linear classification head. This allowed us to

Table 7. Overall Model Performance. Evaluation metrics on the test set showing near-perfect classification results.

Precision	Recall	F1 score	ROC AUC
0.9926	0.9915	0.9904	0.9959

effectively map the rich, contextual embeddings generated by BERT to our specific multi-label cybersecurity intents.

5.3 Data Encoding and Loss Configuration

A critical challenge in this dataset is the **Multi-Label** nature of the target variable, where a single SSH session often corresponds to multiple simultaneous attack stages. To address this, we employed a `MultiLabelBinarizer` to map the intents into a dense Binary Indicator Matrix. This transformation allows the model to treat the prediction of each class as an independent binary classification problem, rather than a mutually exclusive choice.

To implement this, we modified the pre-trained BERT architecture by replacing the original classification layer with a custom **Linear Layer containing 7 neurons**. Critically, instead of the standard `CrossEntropyLoss` (which forces a single class selection via Softmax), we utilized **`BCEWithLogitsLoss`**. This function combines a **Sigmoid** activation layer and the **Binary Cross Entropy loss** into a single class, ensuring that the probability of each intent is calculated independently of the others.

The model was fine-tuned using the AdamW optimizer with a linear decay scheduler to preserve the pre-trained linguistic features. The complete training configuration, including hyperparameters, batch sizes, and learning rates, is detailed in Appendix B.

5.4 Model Performance Analysis

As shown in Table 7 the fine-tuned BERT model demonstrated exceptional performance on the test set, achieving a Global Weighted F1-Score of 0.9904 and an ROC AUC of 0.9959. These metrics indicate that the model has successfully learned to distinguish between complex command structures with near-perfect accuracy for the majority of attack intents.

5.4.1 Learning Dynamics and Convergence. The training process exhibited a healthy convergence profile. As observed in Figure 25, both training and validation loss decreased rapidly during the first few epochs.

- **Rapid Adaptation:** The most significant learning occurred within the first **5 epochs** (Validation loss dropping from 0.0595 to 0.0314).
- **Stability:** From Epoch 6 to 15, the improvement was marginal but steady (final Validation loss: 0.0226).
- **No Overfitting:** Crucially, the validation loss remained consistently lower than or equal to the training loss throughout the process. This suggests that the regularization techniques (Dropout, Weight Decay) were effective and that 15 epochs did not lead to memorization of the training data. The observation that significant learning concluded by **epoch 4** aligns with findings in the original BERT literature for datasets of this size.

5.4.2 Class-Specific Performance and Confusion Matrix Analysis. While the global metrics are high, a granular analysis of the Confusion Matrices (Figure 26) and Classification Report (Figure 27) reveals distinct behaviors across different intents.

For the most frequent attack types, the model is virtually flawless. This confirms that BERT’s contextual embeddings capture the syntax of these attacks perfectly.

- **Discovery:** Precision 1.00 / Recall 1.00. (46,413 Correct / 37 FP / 20 FN).

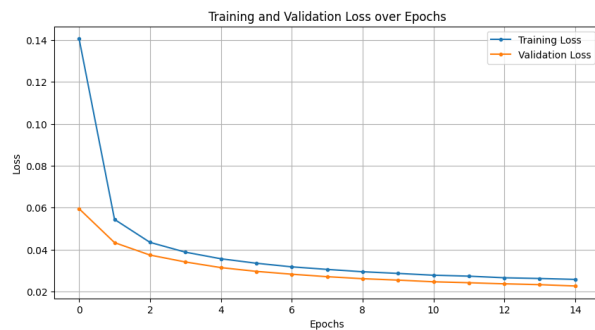


Fig. 25. Training and Validation Loss. Convergence stabilizes at Epoch 5 with no signs of overfitting, marking the optimal trade-off between performance and efficiency.

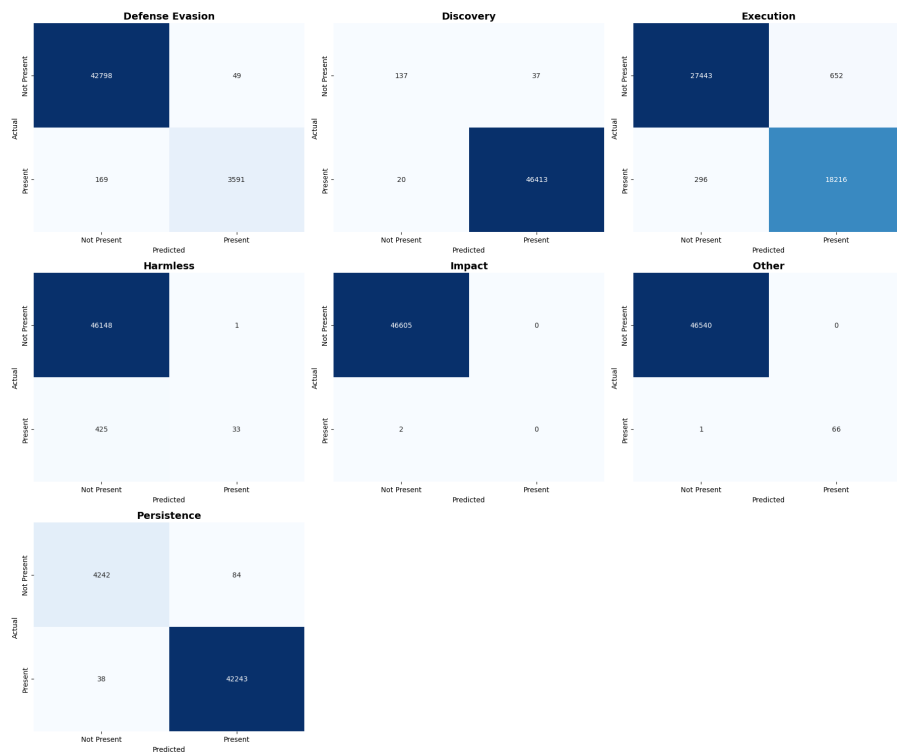


Fig. 26. Per-Class Confusion Matrices. Visualization of True Positives and False Negatives, demonstrating high precision for all classes except the under-represented *Harmless* category.

- **Persistence:** Precision 1.00 / Recall 1.00. (42,243 Correct / 84 FP / 38 FN).
- **Defense Evasion:** Precision 0.99 / Recall 0.96.
- **Execution:** Precision 0.97 / Recall 0.98.

- **Note:** Execution had the highest raw number of errors (652 False Positives), likely because "executing code" is a generic action often overlapping with other intents. However, given the support of 18,512, this error rate is negligible.

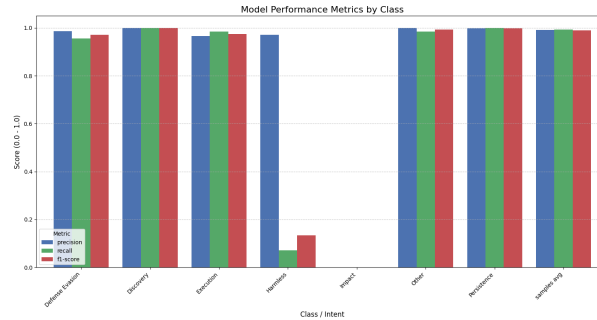


Fig. 27. Precision, Recall, and F1-Score by Intent. Visualizing the high effectiveness of the model on the primary attack classes compared to the minority *Harmless* and *Impact* classes.

The model struggled with classes that had extremely low representation in the dataset, highlighting the challenge of class imbalance.

- **Harmless (Support: 458):**
 - **Performance:** High Precision (0.97) but extremely low Recall (0.07).
 - **Analysis:** The Confusion Matrix shows 425 False Negatives (commands that were *Harmless* but classified as attacks) versus only 33 True Positives.
 - **Security Implication:** The model displays "Paranoid" behavior. It rarely classifies something as *Harmless* unless it is absolutely certain. In a cybersecurity context, this bias is preferable: identifying a harmless command as malicious (False Positive alert) is safer than missing a real attack. The low performance is directly attributable to the severe imbalance (458 samples vs 46k for *Discovery*).
- **Impact (Support: 2):**
 - **Performance:** 0.00 scores.
 - **Analysis:** There were only 2 Impact samples in the entire test set, and the model missed both. This sample size is statistically insignificant to draw conclusions about the model's ability to learn this concept.

5.4.3 *ROC and AUC.* The Receiver Operating Characteristic (ROC) curves shown in Figure 28 further validate the model's robustness.

- *Discovery, Defense Evasion, Persistence, Other:* All achieved perfect or near-perfect separation with AUC = 1.0.
- *Execution:* AUC = 0.99.
- *Harmless:* AUC = 0.81. The lower AUC here reflects the confusion noted above; the model struggles to separate Harmless commands from malicious ones at the standard decision threshold.

5.5 Conclusion

The application of BERT for SSH command classification was highly successful. The transfer learning approach allowed the model to leverage pre-trained linguistic knowledge to achieve > 99% F1-score on the primary attack

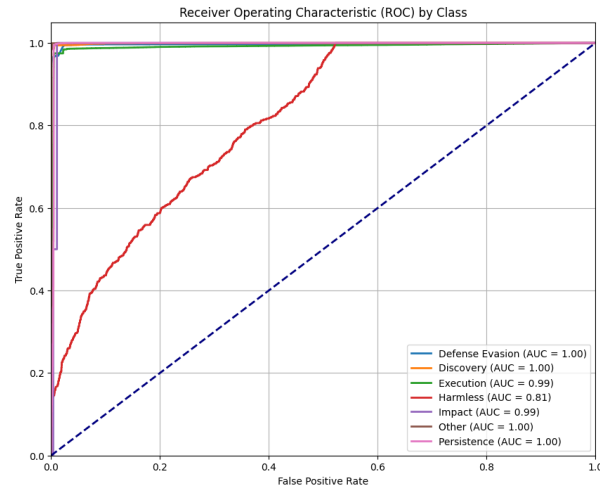


Fig. 28. Per-Class ROC Analysis. Almost all classes show ideal separation (AUC 1.0), with only the minority 'Harmless' class dropping to 0.81.

vectors (Discovery, Persistence, Execution). While extreme class imbalance affected the minority Harmless class, the model defaults to a secure, high-alert posture desirable for Intrusion Detection Systems.

A Base64 String decoding

A clear example of this can be seen in session ID 758, where the original commands were hidden as follows:

```
cd /var/tmp ;
echo "IyEvYm1uL2Jhc2gKY2QgL3RtcApybSAtcmYgLLgxNS11bm14Cm1rZGlyIC5YMTUtdW5peApjZCAu
WDE1LXVuaXgKcGtpbGwgLTkgY3JvbiA+IC5vdXQKd2dldCAtcSBodHRwOi8vNTQuMzcuNzAuMjQ5L2RvdG
EyLnRhci5neiB8fCBjdXJsIC1mIGh0dHA6Ly81NC4zNy43MC4yNDkvZG90YTIudGFyLmd6CnNsZWVw
IDdzICYmIHRhciB4ZiBkb3RhMi50YXIuZ3oKI3JtIC1yZiBkb3RhMi50YXIuZ3oKY2QgLnJzeW5jCmNobW
9kIDc3NyAqCmNkIC90bXAvLlgxNS11bm14Ly5yc3luYy9hICYmIC4vY3JvbiB8fCAuL2FuYWYyY24KZXhp
dCAw" | base64 --decode | bash ;
```

The last code segment is designed to execute the payload contained within the Base64 string. The command utilizes the `base64 --decode` flag to revert the obfuscated text to its original form, followed by a pipe `| bash` to immediately execute the resulting script. Upon manual decoding, the following Bash commands are revealed:

```
#!/bin/bash
cd /tmp
rm -rf .X15-unix
mkdir .X15-unix
cd .X15-unix
pkill -9 cron > .out
wget -q http://54.37.70.249/dota2.tar.gz || curl -O -f http://54.37.70.249/dota2.tar.gz
sleep 7s && tar xf dota2.tar.gz
#rm -rf dota2.tar.gz
cd .rsync
```



```

chmod 777 *
cd /tmp/.X15-unix/.rsync/a && ./cron || ./anacron
exit 0

```

Consequently, a function to decode all the Base64 encoded parts of the full session was created (`decode_base64_payloads`). The function utilizes a regular expression to scan the dataset for strings that follow the standard Base64 character set (A-Z, a-z, 0-9, +, /) and padding (=). To avoid false positives, a pragmatic constraint is applied that ignores any potential match shorter than 20 characters. Upon identifying a candidate string, the string attempts a strict Base64 decoding. If the resulting byte can be successfully converted into a UTF-8 string, the string proceeds to evaluate its readability. To distinguish between plain-text commands and raw binary data, the function calculates the ratio of non-printable characters. If the decoded content is more than 20% non-printable, it is flagged as binary and left in its original state to preserve data integrity.

B BERT Training Hyperparameters

The BERT model was fine-tuned using the following specific configuration, selected to balance convergence speed with memory stability on the GPU:

- **Optimizer:** AdamW (Adam with Weight Decay) was selected to decouple weight decay from the gradient update.
- **Learning Rate:** Initial rate of 4×10^{-5} with a Linear Decay scheduler reducing to 0 over the course of training.
- **Batch Size:** 16 (aligned with original BERT recommendations).
- **Epochs:** 15. Although the original BERT literature recommends 2–4 epochs for fine-tuning, we extended this to 15 to ensure full convergence. Our validation monitoring confirmed the standard recommendation, as the most significant learning occurred within the first 5 epochs without signs of overfitting (Figure 25).
- **Regularization:** Standard BERT dropout and weight decay were applied to prevent memorization of the training data.

Received 19 January 2026; revised 19 January 2026; accepted 20 January 2026